

Advanced Computing Project (2025/26)

Storage system for optimizing LLM checkpointing in HPC

André Miranda
MEI, PG60238

Gustavo Barros
MEI, PG61527

José Soares
MEI, PG60277

Miguel Carvalho
MEI, PG61153

Abstract—Training Large Language Models (LLMs) in High-Performance Computing (HPC) environments is a time-consuming process that requires frequent checkpointing to ensure fault tolerance. However, concurrent writes of massive model states often saturate the shared Parallel File System (PFS), leading to significant I/O contention and training stalls. This paper proposes a policy-driven, two-tier storage orchestration system designed to mitigate the “noisy neighbor” problem in HPC clusters. By transparently redirecting checkpoints to node-local SSDs and employing a centralized orchestrator to manage asynchronous migration to the PFS via a Token Bucket algorithm, we enable granular Quality of Service (QoS) control. Our evaluation on the Deucalion supercomputer demonstrates that

...
Index Terms—High-Performance Computing, Deucalion, A64FX, AMD EPYC 7742, NVIDIA A100, Memory Hierarchy, Large Language Models, Checkpointing, Parallel File Systems, I/O Contention, Quality of Service (QoS), Distributed Systems, Resource Management..

I. INTRODUCTION

The rapid growth of Large Language Models (LLMs) has significantly increased the computational and storage demands of modern High-Performance Computing (HPC) systems. Training such models requires long-running jobs over distributed infrastructures, where fault tolerance becomes essential. Checkpointing—periodically saving the model state—is the standard mechanism to ensure progress is not lost due to failures. However, as model sizes scale to billions of parameters, checkpoint operations generate massive I/O workloads.

In shared HPC environments, these workloads are typically directed to a Parallel File System (PFS), which is simultaneously accessed by multiple users and applications. This leads to contention, particularly when several jobs attempt to write checkpoints concurrently. The resulting “noisy neighbor” effect can degrade performance cluster-wide, increasing latency and even stalling training processes.

Existing approaches either optimize checkpoint size or leverage faster intermediate storage, but often lack coordination mechanisms to regulate global I/O pressure. This paper proposes a policy-driven, two-tier storage orchestration system that combines node-local SSD staging with centralized rate control. By decoupling checkpoint generation from persistent storage and introducing a Token Bucket-based migration strategy, the system provides predictable Quality of Service (QoS) while maintaining transparency to user applications.

We evaluate the proposed system on the Deucalion supercomputer, demonstrating its ability to reduce I/O contention

and improve overall system stability during large-scale LLM training.

II. BACKGROUND AND MOTIVATION

A. Checkpointing in HPC

Checkpointing is a fundamental technique for fault tolerance in High-Performance Computing (HPC) applications. Since large-scale training jobs may execute for hours or even weeks, failures caused by hardware faults, network instability, or software errors can lead to significant computational losses. To mitigate this problem, applications periodically save their execution state to stable storage, enabling recovery from the most recent checkpoint instead of restarting from the beginning.

In distributed deep learning workloads, checkpoints typically include model parameters, optimizer states, gradients, and training metadata. The rapid growth of Large Language Models (LLMs) has significantly increased checkpoint sizes, often reaching tens or hundreds of gigabytes per save operation. As a result, checkpointing has evolved into a major source of I/O pressure in modern HPC systems.

B. Parallel File Systems and Contention

Parallel File Systems (PFSs) such as Lustre are designed to provide high aggregate throughput and scalable shared storage for HPC environments. These systems distribute data across multiple storage targets and metadata servers, enabling concurrent access from thousands of compute nodes.

Despite their scalability, PFSs remain vulnerable to bursty and highly synchronized workloads. In deep learning environments, checkpoint operations are often triggered simultaneously across distributed jobs, generating sudden I/O spikes that overwhelm storage resources. This behavior increases latency, reduces effective throughput, and creates contention between unrelated applications sharing the same filesystem.

This phenomenon is commonly referred to as the “noisy neighbor” problem, where the I/O activity of a subset of jobs negatively impacts the performance of others. As LLM training scales further, these interference effects become increasingly problematic for shared HPC infrastructures.

C. Motivation

The main motivation of this work is that checkpointing is not latency-critical for persistence, but it is latency-critical for training continuation.

In practice, training can resume as soon as checkpoint data is safely written to fast temporary storage, while durable persistence to the PFS can occur asynchronously.

This distinction enables the decoupling of checkpoint generation from long-term storage operations. By leveraging node-local SSDs as an intermediate staging layer and regulating background transfers to the PFS, it becomes possible to reduce contention, smooth I/O bursts, and provide predictable Quality of Service (QoS) across concurrent workloads.

The proposed approach therefore aims to preserve fast checkpoint completion from the application’s perspective while improving overall storage stability at the cluster level.

III. SYSTEM DESIGN

The proposed system adopts a two-tier storage architecture combined with a centralized orchestration mechanism to regulate checkpoint traffic in shared HPC environments.

The main design goals are:

- Reduce checkpoint latency perceived by training applications
- Minimize congestion on the shared Parallel File System (PFS)
- Provide fairness and QoS isolation between concurrent jobs
- Preserve transparency and compatibility with existing HPC workflows

Instead of writing checkpoints directly to the PFS, the system temporarily stages data in node-local SSDs and asynchronously migrates it under centralized control. This decouples checkpoint generation from persistent storage operations, reducing synchronized I/O bursts.

A. Design Overview

The system consists of three main components:

- **Interception Layer:** Transparently redirects checkpoint writes to local storage.
- **Local Storage Tier:** Node-local SSDs for fast staging.
- **Control Plane:** Central orchestrator managing global I/O behavior and enforcing QoS policies.

When a checkpoint operation is triggered, the application writes data to the local SSD instead of directly accessing the PFS. Once the local write completes, the application can immediately resume execution. Checkpoints are later transferred to persistent storage through controlled background migration managed by the control plane.

This design minimizes training stalls while smoothing I/O pressure across the cluster.

B. I/O Interception Mechanism

The interception layer operates in user space and transparently captures checkpoint-related POSIX I/O operations. Using dynamic linking techniques, file write operations are redirected from the shared PFS to node-local storage without requiring modifications to applications or deep learning frameworks.

From the application’s perspective, checkpoint semantics remain unchanged. This preserves compatibility with existing

HPC software stacks, schedulers, and distributed training frameworks.

By intercepting writes before they reach the PFS, the system prevents synchronized checkpoint bursts from immediately impacting shared storage resources.

C. Local Storage Tier

Each compute node provides local SSD storage used as a temporary checkpoint staging area. Compared to remote PFS access, local SSDs offer significantly lower latency and higher write throughput.

This storage tier absorbs bursty checkpoint traffic and enables rapid checkpoint completion, allowing training processes to continue execution without waiting for persistence to complete.

In addition to storing checkpoint data, the local tier maintains metadata describing:

- Checkpoint identifiers
- Migration status
- File integrity information
- Persistence progress

The local storage layer therefore acts as a buffering mechanism between high-frequency checkpoint generation and comparatively slower shared storage operations.

D. Control Plane

The control plane is responsible for globally coordinating checkpoint migration across compute nodes. It maintains a system-wide view of:

- Active jobs
- Pending checkpoint queues
- PFS utilization
- Allocated bandwidth quotas

Rather than allowing unrestricted checkpoint flushing, the control plane regulates migration traffic using a Token Bucket-based QoS mechanism. Each job receives a configurable bandwidth allocation that determines the rate at which checkpoint data may be transferred to the PFS.

This centralized coordination prevents individual jobs from monopolizing storage bandwidth and mitigates the noisy neighbor effect commonly observed in shared HPC systems.

By smoothing bursty migration traffic over time, the control plane improves overall storage stability and fairness while maintaining predictable checkpoint behavior.

IV. QoS-AWARE DATA MIGRATION

A. Token Bucket Model

To regulate PFS access, each job is assigned a token bucket defining its allowed write rate. Tokens accumulate over time and are consumed during data transfers.

B. Scheduling Policy

The orchestrator schedules checkpoint flushes based on:

- Token availability
- Queue size
- System load

C. Fairness and Isolation

The Token Bucket mechanism ensures that no job exceeds its allocated bandwidth, providing fairness and mitigating interference.

V. IMPLEMENTATION

A. Prototype Design

The system is implemented as a user-space middleware layer, avoiding kernel modifications. The interception layer is realized using dynamic linking techniques.

B. Data Management

Checkpoint files are stored locally with metadata tracking their migration status. A background daemon handles transfers to the PFS.

C. Fault Tolerance

To ensure reliability, the system maintains redundancy metadata and guarantees that checkpoints are eventually persisted to the PFS.

D. Integration with HPC Workflows

The system is compatible with existing job schedulers and deep learning frameworks, requiring no changes to user applications.

VI. BENCHMARKING

Experiments were conducted on the Deucalion supercomputer, featuring heterogeneous compute nodes and a shared PFS. Workloads simulate LLM checkpointing behavior with varying sizes and frequencies.

A. Workloads

We evaluate the system using synthetic and realistic checkpointing workloads designed to mimic Large Language Model (LLM) training behavior:

- TODO

B. Metrics

- Checkpoint latency
- Training stall duration
- PFS throughput and variability
- Fairness index across jobs
- I/O congestion
- CPU/GPU and memory usage

VII. RELATED WORK

The optimization of checkpointing in High-Performance Computing (HPC) has been extensively studied, primarily focusing on reducing write latency and mitigating Parallel File System (PFS) congestion.

A. Architectural and Tiering Approaches

Several systems utilize hierarchical storage to hide I/O latency. **Check-N-Run** focuses on staging checkpoints in local node resources (DRAM or SSDs), allowing training to resume immediately while data is moved to the PFS in the background. However, it lacks a global coordination mechanism to prevent PFS saturation during the background flush. **DeepFreeze** (similar to Monarch) employs transparent storage tiering to accelerate the write path by abstracting complex local tiers like NVMe or PMEM from the user. While effective at the node level, these systems often ignore the "noisy neighbor" effect in shared environments.

B. Adaptive and Optimization Techniques

Other research focuses on the frequency and size of checkpoints. **CheckFreq** introduces an algorithmic approach to identify optimal checkpoint boundaries and pipelines the process with training iterations, dynamically adjusting frequency to maintain overhead below a threshold. **BitSnap** addresses the data volume by employing sparsification and quantization, reducing checkpoint sizes from 16-bit to 4-bit weights before they reach the I/O layer. While these reduce total I/O pressure, they do not manage the storage hierarchy or provide holistic Quality of Service (QoS) guarantees.

C. Software-Defined Storage and QoS Control

Our work is heavily informed by **PADLL**, a storage middleware that enables application-agnostic QoS control for both data and metadata workflows. PADLL utilizes a decoupled Software-Defined Storage (SDS) architecture, where data plane stages intercept POSIX calls and a centralized control plane coordinates I/O rates across all jobs. While PADLL is a general-purpose QoS framework, our system specializes this paradigm for the specific lifecycle of LLM checkpoints—integrating node-local SSD staging with PADLL-inspired Token Bucket rate limiting to ensure cluster harmony during massive model state migrations.

VIII. RESULTS

Colocar resultados em geral e em específico para cada métrica?

IX. DISCUSSION

A. Trade-offs

B. Scalability

C. Limitations

X. CONCLUSIONS

TODO

XI. FUTURE WORK

Future work will focus on...

TODO

REFERENCES

- [1] EuroHPC Joint Undertaking, “Access Calls – Technical Guidelines, Version 6 (26.05.2025),” 2025. [Link](#)
- [2] Fujitsu Ltd., “FUJITSU Processor A64FX – Architecture and Specifications,” 2020. [Link](#)
- [3] AMD, “AMD EPYC 7742 Processor (Rome) – Product Specifications and Performance Guide,” 2019–2024. [Link](#)